

Hands-on 4

Difference between JPA, Hibernate and Spring Data JPA

1. Introduction

In enterprise Java development, applications rarely execute SQL statements manually. Instead, developers use Object Relational Mapping (ORM), where Java objects are automatically mapped to relational database tables. Three technologies dominate this ecosystem: JPA, Hibernate, and Spring Data JPA. Although these names are often used interchangeably, they represent different layers of the persistence architecture. Understanding their responsibilities is essential for Java Full Stack interviews and Spring Boot development.

2. Object Relational Mapping (ORM)

ORM is a programming technique that bridges the gap between object-oriented programming and relational databases. Instead of writing SQL for every CRUD operation, developers work with Java objects while the ORM framework generates SQL automatically.

Java Object		Database Table
-----		-----
Country object	----->	country table
country.getName()		SELECT name FROM country

3. Java Persistence API (JPA)

JPA (Java Persistence API) is a specification published by Jakarta EE. A specification is a contract that defines how persistence should work. JPA provides interfaces, annotations, lifecycle rules, and APIs such as EntityManager. It does not communicate with the database itself and therefore cannot be used without an implementation.

Common JPA Annotations

```
@Entity
@Table(name="country")
public class Country {

    @Id
    private String code;

    @Column(name="name")
    private String name;
}
```

The annotations above come from JPA. They describe how a Java class maps to a database table but they do not execute SQL.

4. Hibernate

Hibernate is an ORM framework and the most widely used implementation of JPA. It understands JPA annotations and translates Java operations into SQL queries. Hibernate manages sessions, transactions, caching, lazy loading, relationship mapping, dirty checking, and query generation.

Example

```
Country country = new Country("IN", "India");
entityManager.persist(country);

// Generated SQL
INSERT INTO country(code, name)
VALUES('IN', 'India');
```

Similarly, calling `findById()` or `findAll()` causes Hibernate to generate `SELECT` statements. Hibernate allows developers to concentrate on business logic instead of SQL generation.

5. Spring Data JPA

Spring Data JPA is a Spring module that sits above JPA. Instead of writing DAO classes manually, developers simply create repository interfaces. Spring generates the implementation automatically at runtime.

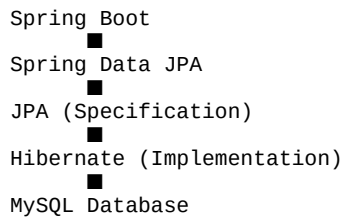
```
public interface CountryRepository
    extends JpaRepository<Country, String> {

    List<Country> findByNameContaining(String name);
}
```

Without writing SQL or implementation classes, methods such as `save()`, `findAll()`, `deleteById()`, `findById()` and custom derived queries become available automatically.

6. Relationship among the Three

These technologies work together rather than compete.



Spring Boot configures Spring Data JPA automatically. Spring Data JPA calls JPA APIs. Hibernate implements those APIs and finally communicates with MySQL.

7. Code Comparison

Using Hibernate Directly

```
Session session = sessionFactory.openSession();
Transaction tx = session.beginTransaction();

session.save(country);

tx.commit();
session.close();
```

Using Spring Data JPA

```
@Autowired
CountryRepository repository;

repository.save(country);
```

Spring Data JPA internally uses Hibernate. Therefore, developers write significantly less code while retaining Hibernate's powerful ORM features.

8. Advantages

JPA provides portability because the application can switch implementations with minimal code changes. Hibernate offers advanced ORM capabilities and performance optimizations. Spring Data JPA improves productivity by removing boilerplate repository code and integrating seamlessly with Spring Boot.

9. Feature Comparison

	JPA Specification only	Defines annotations and interfaces	No implementation	Hibernate ORM Framework	Implements JPA	Generates SQL	Communicates with database	Spring Data JPA	Spring Framework module	Built on top of JPA	Provides JpaRepository	Automatic CRUD methods
--	------------------------	------------------------------------	-------------------	-------------------------	----------------	---------------	----------------------------	-----------------	-------------------------	---------------------	------------------------	------------------------

10. Real Project Example

```
@Entity
class Country { ... }

public interface CountryRepository
    extends JpaRepository<Country,String> {}

@Autowired
CountryRepository repository;

List<Country> list = repository.findAll();
Country c = repository.findById("IN").orElse(null);
```

In this example: JPA provides @Entity and @Id annotations. Hibernate converts repository calls into SQL. Spring Data JPA provides repository methods.

11. Conclusion

JPA, Hibernate, and Spring Data JPA are complementary technologies. JPA defines the persistence standard, Hibernate implements that standard, and Spring Data JPA simplifies application development by generating repository implementations automatically. In modern Spring Boot applications, developers generally interact with Spring Data JPA, while Hibernate works behind the scenes to execute SQL statements and manage persistence.